

Sharing software environments

Harmen Stoppels · Spack maintainer
Leibniz-Rechenzentrum · 15 October 2026

INTRODUCTION

Sharing a software environment

- A researcher / colleague wants to **reproduce** your results
- You move from a **laptop or VM** to a cluster
- You build a stack in CI for your users
- The same stack on **several clusters**, or on short-lived cloud instances
- A **container** for Kubernetes, JupyterHub, Apptainer…

> Many ways in which you can share an "environment": a list of packages, exact versions, binaries, or a whole file system.

INTRODUCTION

Spack offers various ways to share software stacks

`spack.yaml`

abstract specs + config, like `hdf5@1.14`

WHEN TO USE

different machines

`spack.lock`

concrete dependency graph: all resolved packages, their versions, variants, architecture, platform.

WHEN TO USE

~identical machines

`build cache`

binaries archived as `*.tar.gz`

WHEN TO USE

compatible machines

`container image`

binaries + OS user land

WHEN TO USE

deployment, sharing with non-Spack users

There are other ways I won't cover in this talk, namely central installations with

- environment modules (`module load hdf5`)
- Spack environment views (like Python virtual environments)
- an upstream database for Spack users

1

Spack environments

ENVIRONMENTS RECAP

`spack.yaml` and `spack.lock`

`spack.yaml`: input for the solver

```
spack:
  specs:
  - hdf5+mpi
  - mpich
  - py-h5py
  view: false
  concretizer:
    unify: true
```

Abstract specs + configuration. Meant to be human-readable.

`spack.lock`: input for the installer

```
{"roots": [{"hash": "jceyzq7…", "spec": "hdf5+mpi"}, …],
 "concrete_specs": {
  "jceyzq7…": {"name": "hdf5", "version": "1.14.6",
    "arch": {"platform_os": "ubuntu26.04",
             "target": "x86_64_v3"},
    "dependencies": [{"name": "cmake", "hash": "a4wqsr7…",
                      "parameters": {"deptypes": ["build"]}},
                     …]},
  "yc4n2pp…": {"name": "glibc", "version": "2.43",
    "external": {"path": "/usr"}, …},
  …59 nodes…}}
```

The fully concrete DAG: every version, variant, target and dependency, keyed by hash. Generated by `spack concretize`.

```
$ spack env create my-env
$ spack -e my-env add hdf5+mpi
$ spack -e my-env concretize   # creates spack.lock
$ spack -e my-env install
```

Both `spack.yaml` and `spack.lock` can be shared

`spack.yaml`: input for the solver

```
spack:
  specs:
  - hdf5+mpi
  - mpich
  - py-h5py
  view: false
  concretizer:
    unify: true
```

```
$ spack env create myenv spack.yaml
$ spack -e my-env concretize
$ spack -e my-env install
```

`spack.lock`: input for the installer

```
{"roots": [{"hash": "jceyzq7…", "spec": "hdf5+mpi"}, …],
 "concrete_specs": { …]},
```

```
$ spack env create myenv spack.lock
$ spack -e myenv install
```

spack.yaml is "portable"

On another machine, re-concretization uses *that* machine's:

- Spack version and **package repository** commit
- compilers and **externals** (`packages.yaml`)
- OS, CPU target, available binaries (reuse)

So the result is "the same software, appropriate for this machine".

You can pin the things you care about:

```yaml
spack:
  specs: [hdf5+mpi, mpich, py-h5py]
  repos:
    builtin:
      commit: 6427933d…  # or tag/branch
  packages:
    all:
      require:
      - target=x86_64_v3 # don't tune for my laptop
    hdf5:
      require: "@1.14"
```

> Good for "same software". Not good for "exactly what I ran last month".

spack.lock is machine-specific

```
laptop:hpc-env $ # spack.yaml: what we asked for
laptop:hpc-env $ cat spack.yaml
spack:
  specs:
  - hdf5+mpi
  - mpich
  - py-h5py
  view: false
  concretizer:
    unify: true
laptop:hpc-env $ # spack.lock: what we got. It's just JSON
laptop:hpc-env $ jq .roots spack.lock
[
  {
    "hash": "jceyzq7qvavwbbfywa3dt3jfurkfk4ba",
    "spec": "hdf5+mpi"
  },
  {
    "hash": "xkilhym6dn2n7afhh5dtjskmj5jwjbhm",
    "spec": "mpich"
  },
  {
    "hash": "qaefg7qzt6uinhgkwad3wkjjd5hhozmj",
    "spec": "py-h5py"
  }
]
laptop:hpc-env $ # two nodes in the lock file were not built by Spack
laptop:hpc-env $ spack -e . find --external --paths
==> In environment /home/bot/lrz-demo/envs/hpc (3 root specs)
[+] hdf5+mpi  [+] mpich  [+] py-h5py

-- linux-ubuntu26.04-x86_64_v3 / no compilers ------------------
gcc@15.2.0  /usr
glibc@2.43  /usr
==> 2 installed packages
==> 0 concretized packages to be installed (show with `spack find -c`)
laptop:hpc-env $
```

spack.lock is machine-specific

Machine specific `target=znver3` or `target=cannonlake`

Externals are part of the DAG and of every dependent's hash. They record a **prefix on this machine**.

This makes sharing difficult across different machines.

Typical externals on HPC systems:

libc	**glibc: always**
compiler	system / vendor gcc, oneAPI, nvhpc
MPI	vendor MPI tuned for the fabric
GPU	CUDA, ROCm, Level Zero drivers
scheduler	Slurm, PMIx, libfabric/UCX providers

```
packages:
  mpi:
    buildable: false
  intel-oneapi-mpi:
    externals:
    - spec: intel-oneapi-mpi@2021.14
      prefix: /opt/intel/oneapi
  slurm:
    externals:
    - spec: slurm@24.05
      prefix: /usr
```

LIBC

glibc is more or less a required external

- Since Spack v1.0, libc is an **explicit node** in the DAG: `[e] glibc@2.43`
- Binaries link against symbol versions such as `GLIBC_2.38`. They run on a glibc that is **at least as new**.
- The concretizer only **reuses** a binary if it is libc-compatible with the host

> **Build on the oldest glibc you need to support.**

SYSTEM	GLIBC
SLES 15 SP3–SP5	2.31
SLES 15 SP6–SP7	2.38
RHEL / Rocky 8	2.28
RHEL / Rocky 9	2.34
Ubuntu 24.04	2.39
Ubuntu 26.04 (my laptop)	2.43

LIBC · RECORDED

Error you see when glibc is too old

```
laptop:hpc-env $ # lstopo from hwloc, built with Spack on this laptop
laptop:hpc-env $ ldd --version | head -1
ldd (Ubuntu GLIBC 2.43-2ubuntu2.4) 2.43
laptop:hpc-env $ lstopo=$(spack -e . location -i hwloc)/bin/lstopo
laptop:hpc-env $ $lstopo --version
lstopo 2.13.0
laptop:hpc-env $ # the very same file on SLES 15 SP6 (mounted at the same path)
laptop:hpc-env $ docker run --rm -v ~/lrz-demo:$HOME/lrz-demo registry.suse.com/bci/bci-base:15.6 ldd --versio
n | head -1
ldd (GNU libc) 2.38
laptop:hpc-env $ docker run --rm -v ~/lrz-demo:$HOME/lrz-demo registry.suse.com/bci/bci-base:15.6 $lstopo --ve
rsion
/home/bot/lrz-demo/store/…/hwloc-2.13.0-ujctyey/bin/lstopo: /lib64/libm.so.6: version `GLIBC_2.43' not found (
required by /home/bot/lrz-demo/store/…/hwloc-2.13.0-ujctyey/bin/lstopo)
/home/bot/lrz-demo/store/…/hwloc-2.13.0-ujctyey/bin/lstopo: /lib64/libc.so.6: version `GLIBC_2.42' not found (
required by /home/bot/lrz-demo/store/…/ncurses-6.6-ekvivpv/lib/libtinfo.so.6)
laptop:hpc-env $
```

Build caches

```
$ spack mirror add --unsigned mycache file:///shared/cache
$ spack buildcache push --update-index mycache
# elsewhere: same mirror config + spack.lock
$ spack install       # fetch binaries, no builds
```

- One tarball per package (+ spec metadata), on a file system, S3, HTTP, or an **OCI registry**
- `--update-index` rebuilds the index used for **reuse** by the concretizer
- Signed by default (`spack gpg`); `--unsigned` for trusted locations

Relocation

Binaries embed absolute paths (RPATHs, scripts, config files). On install, Spack rewrites them to the new prefix. Strings in binaries can only **shrink**, so build with a padded install root:

```
config:
  install_tree:
    root: /opt/spack
    padded_length: 128
```

Two ways to use a build cache

Build cache + `spack.yaml` (reuse concretization)

```
# both machines configure the same mirror
$ spack env create myenv spack.yaml
$ spack concretize   # reuses compatible binaries
$ spack install      # builds the rest
```

- Each machine concretizes for **itself**; the cache is a source of reusable binaries
- Reusable specs are **installed from the build cache**, others **built from source**
- Works well when B has a **glibc** version ≥ than A

> As much reuse as possible.

Build cache + `spack.lock`

```
# reproduce A's DAG on B without source builds
$ spack env create myenv spack.lock
$ spack install --cache-only
```

- Machine B installs the **exact* same binaries** built on A
- Works when machine B is equivalent to A: CPU target, externals
- * "Exact" up to **relocation**: a different install root means rewritten paths in binaries and scripts

> The same binaries, installed by Spack, on a compatible host.

Push on one machine, install from the lock file on another

```
laptop:hpc-env $ # push the installed environment to a build cache on a shared file system
laptop:hpc-env $ spack -e . buildcache push --unsigned --without-build-dependencies ~/lrz-demo/cache
==> Selected 32 specs to push to file:///home/bot/lrz-demo/cache
==> [ 1/32] Pushed pigz@2.8/afklka7
==> [ 2/32] Pushed libmd@1.1.0/yvvn7mu
==> [ 3/32] Pushed tar@1.35/f5xe4px
==> [ 4/32] Pushed libbsd@0.12.2/7o3mv6c
==> [ 5/32] Pushed zstd@1.5.7/63aruxk
==> [ 6/32] Pushed util-linux-uuid@2.41/clufswx
==> [ 7/32] Pushed sqlite@3.53.1/unjd5da
==> [ 8/32] Pushed readline@8.3/k25xiih
==> [ 9/32] Pushed openssl@3.6.1/33yozp5
==> [10/32] Pushed libffi@3.5.2/aiceosz
==> [11/32] Pushed gettext@1.0/sle3ix4
==> [12/32] Pushed gdbm@1.26/cq4dkuc
: 33 lines omitted :
[+] clufswx util-linux-uuid@2.41 /home/bot/lrz-demo/b/…/util-linux-uuid-2.41-clufswx (0s)
[+] hhjyyqy xz@5.8.3 /home/bot/lrz-demo/b/…/xz-5.8.3-hhjyyqy (0s)
[+] qzkk5ym libpciaccess@0.17 /home/bot/lrz-demo/b/…/libpciaccess-0.17-qzkk5ym (0s)
[+] cbtgjrh bzip2@1.0.8 /home/bot/lrz-demo/b/…/bzip2-1.0.8-cbtgjrh (0s)
[+] yvl6jpi pkgconf@2.5.1 /home/bot/lrz-demo/b/…/pkgconf-2.5.1-yvl6jpi (0s)
[+] g72d7i3 zlib-ng@2.3.3 /home/bot/lrz-demo/b/…/zlib-ng-2.3.3-g72d7i3 (0s)
[+] vbwvgwx libiconv@1.18 /home/bot/lrz-demo/b/…/libiconv-1.18-vbwvgwx (0s)
[+] 63aruxk zstd@1.5.7 /home/bot/lrz-demo/b/…/zstd-1.5.7-63aruxk (0s)
[+] 7o3mv6c libbsd@0.12.2 /home/bot/lrz-demo/b/…/libbsd-0.12.2-7o3mv6c (0s)
[+] afklka7 pigz@2.8 /home/bot/lrz-demo/b/…/pigz-2.8-afklka7 (0s)
[+] ujlg2ua libxml2@2.15.3 /home/bot/lrz-demo/b/…/libxml2-2.15.3-ujlg2ua (1s)
[+] d462mhw expat@2.8.1 /home/bot/lrz-demo/b/…/expat-2.8.1-d462mhw (1s)
[+] f5xe4px tar@1.35 /home/bot/lrz-demo/b/…/tar-1.35-f5xe4px (1s)
[+] 7kdghmr openblas@0.3.33 /home/bot/lrz-demo/b/…/openblas-0.3.33-7kdghmr (1s)
[+] 33yozp5 openssl@3.6.1 /home/bot/lrz-demo/b/…/openssl-3.6.1-33yozp5 (1s)
[+] ekvivpv ncurses@6.6 /home/bot/lrz-demo/b/…/ncurses-6.6-ekvivpv (1s)
[+] k25xiih readline@8.3 /home/bot/lrz-demo/b/…/readline-8.3-k25xiih (0s)
[+] ujctyey hwloc@2.13.0 /home/bot/lrz-demo/b/…/hwloc-2.13.0-ujctyey (0s)
[+] cq4dkuc gdbm@1.26 /home/bot/lrz-demo/b/…/gdbm-1.26-cq4dkuc (1s)
[+] unjd5da sqlite@3.53.1 /home/bot/lrz-demo/b/…/sqlite-3.53.1-unjd5da (1s)
[+] sle3ix4 gettext@1.0 /home/bot/lrz-demo/b/…/gettext-1.0-sle3ix4 (1s)
[+] xkilhym mpich@5.0.1 /home/bot/lrz-demo/b/…/mpich-5.0.1-xkilhym (2s)
[+] jceyzq7 hdf5@1.14.6 /home/bot/lrz-demo/b/…/hdf5-1.14.6-jceyzq7 (1s)
[+] derywaq python@3.14.5 /home/bot/lrz-demo/b/…/python-3.14.5-derywaq (2s)
[+] j76kyd4 python-venv@1.0 /home/bot/lrz-demo/b/…/python-venv-1.0-j76kyd4 (0s)
[+] evb4x7y py-numpy@2.4.6 /home/bot/lrz-demo/b/…/py-numpy-2.4.6-evb4x7y (1s)
[+] tjrgxz6 py-mpi4py@4.1.1 /home/bot/lrz-demo/b/…/py-mpi4py-4.1.1-tjrgxz6 (1s)
[+] qaefg7q py-h5py@3.16.0 /home/bot/lrz-demo/b/…/py-h5py-3.16.0-qaefg7q (1s)
==> Updating view at /home/bot/lrz-demo/machines/b/my-env/.spack-env/view
machine-b $
```

SHARING TIPS

Keeping environments shareable

Config includes can help

```
spack:
  specs: [hdf5+mpi, mpich, py-h5py]
  include:
  - path: ./site/cluster-a.yaml
    when: hostname.startswith("login")
  - path: ./site/laptop.yaml
    when: platform == "darwin"
```

Site specifics (externals, compilers, install root) go in included site scopes, not in `spack.yaml`.

Using few externals

- **Just let Spack build things**, including `cmake`, `python`, `zlib`, …
- **Use externals when necessary**: closed-source libfabric, Slurm/PMIx, ...
- I personally don't use `spack external find`

How other tools handle "externals"

EESSI (CVMFS) / Nix / Guix

Ship their **own libc** (or libcs) at a particular absolute path like

- `/cvmfs/foo.cern.ch/...`
- `/nix/store/...`
- `/gnu/store/...`

Conda / pip

Link against old glibc and assume compatibility, largely similar to Spack.

Spack v1.4?

I'm looking at a way to make glibc installable like any other package, which would eliminate the remaining external; we'd support both approaches.

2

Container images using `spack containerize`

WHAT YOU KNOW

Build with Dockerfile

```
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y gcc
COPY app.c /src/
RUN gcc -o /usr/bin/app /src/app.c
```

```
$ docker build -t myorg/app:1.0 .
$ docker push myorg/app:1.0
```

```
diff after RUN gcc …
```

```
diff after COPY
```

```
diff after RUN apt-get …
```

```
ubuntu:24.04 layers
```

- Each step runs **inside** a container of the previous state.
- File system changes are captured as a new layer (available thanks to overlayfs).

Generate a Dockerfile from an environment

```yaml
spack:
  specs:
  - hdf5+mpi
  - mpich
  - py-h5py
  packages:
    all:
      require: [target=x86_64_v3]
  container:
    format: docker    # or singularity
    images:
      os: ubuntu:24.04
      spack: "1.2"
```

```
$ spack containerize > Dockerfile
$ docker build -t hpc-env .
```

- `images:`
  - For build and runtime environment
  - Spack-provided images like `spack/ubuntu-noble:1.2`,
  - or custom `build` / `final` images, e.g. an NVIDIA CUDA base with CUDA as an external
- `os_packages:` distro packages for the build or final stage
- `strip`, `labels`
- `format: singularity` writes a `.def` file
- Jinja2 template extension for customization

SPACK CONTAINERIZE · GENERATED (TRIMMED)

A multi-stage build

```
# Build stage with Spack pre-installed and ready to be used
FROM spack/ubuntu-noble:1.2 AS builder

RUN mkdir -p /opt/spack-environment && set -o noclobber \
&&  (echo spack: \
&&   echo '  specs:' \
&&   echo '  - hdf5+mpi' \
     …
&&   echo '  config:' \
&&   echo '    install_tree:' \
&&   echo '      root: /opt/software' \
&&   echo '  view: /opt/views/view') > /opt/spack-environment/spack.yaml

RUN cd /opt/spack-environment && spack env activate . && \
    spack install --fail-fast && spack gc -y

...

# Bare OS image to run the installed executables
FROM ubuntu:24.04
COPY --from=builder /opt/spack-environment /opt/spack-environment
COPY --from=builder /opt/software /opt/software
COPY --from=builder /opt/views /opt/views
...
ENTRYPOINT [ "/entrypoint.sh" ]
CMD [ "/bin/bash" ]
```

- The builder image provides Spack, compilers and the same OS as the final stage
- Note: it ships `spack.yaml`, not `spack.lock`. It **concretizes inside the build**.
- One `RUN` step installs **everything**, so it becomes one big layer
- The final stage copies `/opt/software` and adds an entrypoint that activates the environment

Trade-offs

Pros

- Isolated build environment
- Self-contained, reviewable recipe
- Singularity/Apptainer definition files for HPC
- Can `apt/yum install ...` extras if needed

Cons

- Needs a builder (Docker, Podman, Buildah, Apptainer), not always available (or requires root)
- Everything in one `RUN`: any change to `spack.yaml` rebuilds **everything**; a failure throws away all finished dependencies
- Debugging a failed build inside a container build is painful
- One fat layer: no sharing between different images/tags

Hybrid: give the Dockerfile a build cache

```
# generated Dockerfile, install step edited by hand
RUN --mount=type=bind,from=buildcache,target=/mnt/cache \
    cd /opt/spack-environment && \
    spack env activate . && \
    spack mirror add --unsigned host /mnt/cache && \
    spack install --fail-fast && spack gc -y
```

```
$ docker build \
    --build-context buildcache=/shared/cache \
    -t hpc-env .
```

- A file system build cache on the host is visible **only during that `RUN`**, so it doesn't end up in a layer
- **`spack install` fetches** what it can reuse, builds the rest
- Only binaries compatible with the **builder image** are reused: a cache built on a newer host glibc doesn't help
- Can't easily add to build cache during build (read-only).
- Still one fat layer, still needs a builder

> What if a build cache and a container image were the same thing?

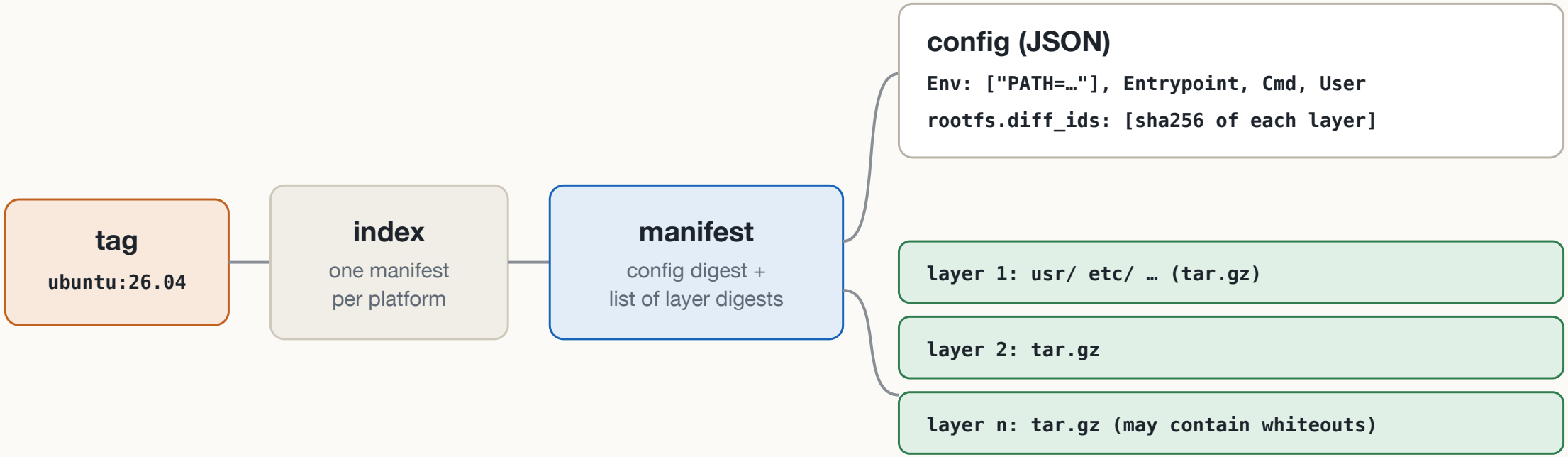
3

Container images as build caches

A bit of container "demystification"

WHAT IS BEHIND A TAG?

A manifest, a config, and tarballs



A tag is the only mutable name. Everything else is content-addressed: blobs are named by their sha256.
A registry is just an HTTP API to put and get blobs and manifests, and to point tags at them.

- `docker pull`: resolve the tag, fetch the manifest and blobs, **unpack the tarballs in order**, start a process with the config's environment
- `docker push`: upload the blobs, then the manifest, then point the tag at it.

It's just files

```
laptop $ # copy an image from Docker Hub into a plain directory (OCI layout)
laptop $ skopeo copy -q docker://ubuntu:26.04 oci:ubuntu:26.04
laptop $ find ubuntu -type f
ubuntu/index.json
ubuntu/blobs/sha256/88a381d5b5ee…
ubuntu/blobs/sha256/06ad70e463aa…
ubuntu/blobs/sha256/4e07a0f12b2c…
ubuntu/blobs/sha256/6e92c4fd8dfd…
ubuntu/oci-layout
laptop $ # index.json points to the manifest, which points to config + layers
laptop $ cd ubuntu/blobs/sha256
laptop $ manifest=$(jq -r '.manifests[0].digest[7:]' ../../index.json)
laptop $ jq 'del(.annotations, .layers[].annotations)' $manifest
{
  "schemaVersion": 2,
: 5 lines omitted :
  },
  "layers": [
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:4e07a0f12b2c…",
      "size": 41837521
    },
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:06ad70e463aa…",
      "size": 388
    }
  ]
}
laptop $ # the config is just JSON
laptop $ jq '.config | {Env, Cmd}' $(jq -r '.config.digest[7:]' $manifest)
{
  "Env": [
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
  ],
  "Cmd": [
    "/bin/bash"
  ]
}
laptop $ # a layer is just a tarball
laptop $ tar -tzf $(jq -r '.layers[0].digest[7:]' $manifest) | grep -E '^(usr/bin/bash|usr/lib/x86_64-linux-gn
u/libc.so.6)$'
usr/bin/bash
usr/lib/x86_64-linux-gnu/libc.so.6
laptop $
```

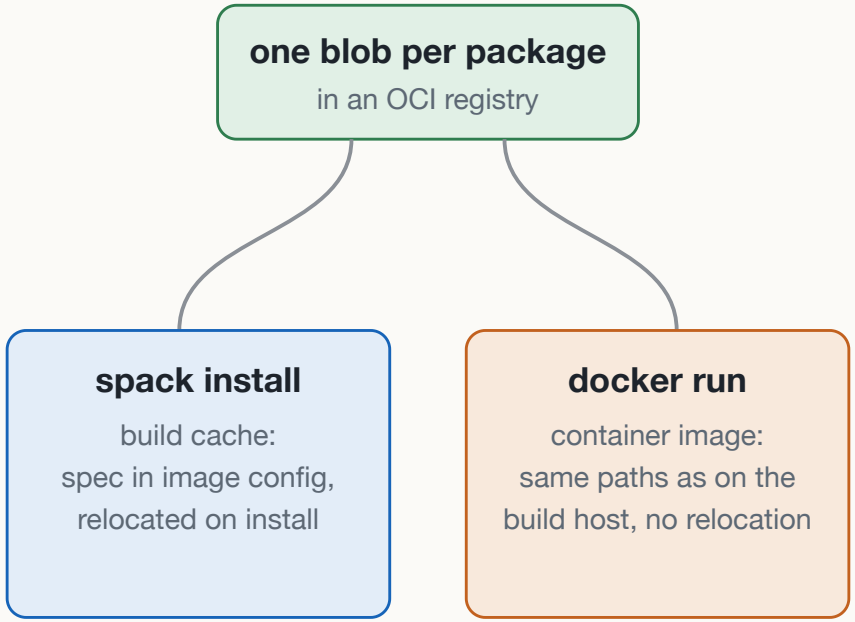
THE KEY OBSERVATION

A Spack build cache tarball is already a layer

A layer is a tarball that gets unpacked at `/`. Spack's build cache tarball of a package contains its files **at their install prefix**:

```
$ curl -s localhost:5000/v2/hpc/blobs/sha256:829c… | tar -tz
home/bot/lrz-demo/store/linux-x86_64_v3/zlib-ng-2.3.3-g72d…/
home/bot/lrz-demo/store/…/zlib-ng-2.3.3-g72d…/include/zlib.h
home/bot/lrz-demo/store/…/zlib-ng-2.3.3-g72d…/lib/libz.so.1
…
```

Unpack it on top of a base image and the package is exactly where it was built. **A build cache is one manifest away from being an image.**



SPACK BUILDCACHE PUSH

Archive what is installed

```
$ spack -e . buildcache push \
      --base-image ubuntu:26.04 \          # provides glibc + the rest of the OS
      --tag hpc-env \                      # one image for all roots + runtime deps
      oci://ghcr.io/myorg/buildcache       # or oci+http://localhost:5000/… for local tests
```

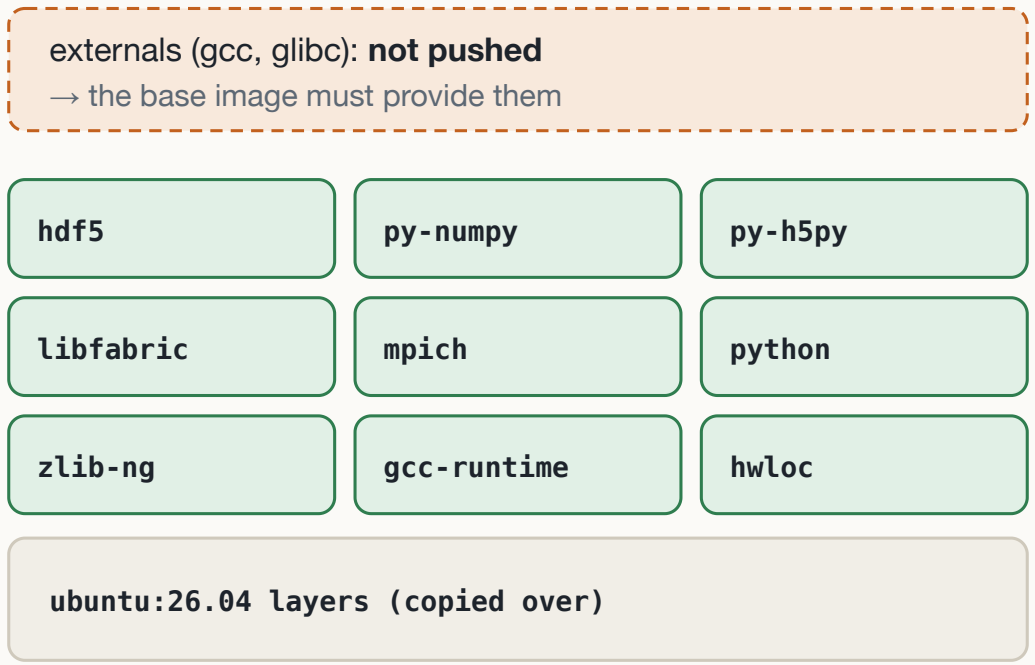
- Input: **installed** specs (built on the host, so easy to debug)
- Output:
  - Container image tagged `ghcr.io/myorg/buildcache:hpc-env`
  - and per-package tags `ghcr.io/myorg/buildcache:name-version-hash.spack`
- No Docker / daemon / root access needed.
- Credentials via env vars: `spack mirror add --oci-username-variable U --oci-password-variable P name oci://…`
- Without `--base-image` you get a "distroless" image: a fine build cache, but not runnable

SPACK BUILDCACHE PUSH · RECORDED

Push installed packages, then `docker run` them

```
laptop:hpc-env $ # a local registry for the demo; any OCI registry works (ghcr.io, GitLab, Harbor, ...)
laptop:hpc-env $ spack -e . mirror add --unsigned hpc-registry oci+http://localhost:5000/hpc
laptop:hpc-env $ # push hdf5 and its runtime dependencies on top of ubuntu:26.04
laptop:hpc-env $ spack -e . buildcache push --without-build-dependencies --base-image ubuntu:26.04 --tag hdf5
hpc-registry hdf5
==> Selected 12 specs to push to oci+http://localhost:5000/hpc
==> Checking for existing specs in the buildcache
==> [ 1/12] Pushed xz@5.8.3/hhjyyqy: sha256:3d312a155b94… (0.02s, 117.78 MB/s)
==> [ 2/12] Pushed libiconv@1.18/vbwvgwx: sha256:9ff5a30cfd5e… (0.02s, 118.43 MB/s)
==> [ 3/12] Pushed ncurses@6.6/ekvivpv: sha256:1eaf3769a182… (0.03s, 195.84 MB/s)
==> [ 4/12] Pushed libxml2@2.15.3/ujlg2ua: sha256:5fd9c3a8c855… (0.02s, 128.03 MB/s)
==> [ 5/12] Pushed libpciaccess@0.17/qzkk5ym: sha256:4522a50170df… (0.03s, 6.25 MB/s)
==> [ 6/12] Pushed libfabric@2.5.1/itb4a2s: sha256:6c462f91dab9… (0.03s, 42.98 MB/s)
==> [ 7/12] Pushed hwloc@2.13.0/ujctyey: sha256:5519992cfa64… (0.03s, 155.24 MB/s)
==> [ 8/12] Pushed zlib-ng@2.3.3/g72d7i3: sha256:829ca9121112… (0.02s, 10.46 MB/s)
: 14 lines omitted :
==> [ 7/12] Tagged hwloc@2.13.0/ujctyey as localhost:5000/hpc:hwloc-2.13.0-ujctyey….spack
==> [ 8/12] Tagged zlib-ng@2.3.3/g72d7i3 as localhost:5000/hpc:zlib-ng-2.3.3-g72d7i3….spack
==> [ 9/12] Tagged pkgconf@2.5.1/yvl6jpi as localhost:5000/hpc:pkgconf-2.5.1-yvl6jpi….spack
==> [10/12] Tagged mpich@5.0.1/xkilhym as localhost:5000/hpc:mpich-5.0.1-xkilhym….spack
==> [11/12] Tagged gcc-runtime@15.2.0/xm76mt3 as localhost:5000/hpc:gcc-runtime-15.2.0-xm76mt3….spack
==> [12/12] Tagged hdf5@1.14.6/jceyzq7 as localhost:5000/hpc:hdf5-1.14.6-jceyzq7….spack
==> Tagged localhost:5000/hpc:hdf5
laptop:hpc-env $ # and run it like any other image
laptop:hpc-env $ docker run --rm localhost:5000/hpc:hdf5 h5dump --version
Unable to find image 'localhost:5000/hpc:hdf5' locally
hdf5: Pulling from hpc
f8eb9ab5b662: Pull complete
1eaf3769a182: Pull complete
6c462f91dab9: Pull complete
5fd9c3a8c855: Pull complete
9ff5a30cfd5e: Pull complete
927452e99f9c: Pull complete
829ca9121112: Pull complete
3d312a155b94: Pull complete
4522a50170df: Pull complete
5fa439fec465: Pull complete
ffcc1d01df12: Pull complete
5519992cfa64: Pull complete
Digest: sha256:6104f123e942…
Status: Downloaded newer image for localhost:5000/hpc:hdf5
h5dump: Version 1.14.6
laptop:hpc-env $ # the lstopo that failed on SLES 15 SP6 works here: the base image brings glibc 2.43
laptop:hpc-env $ docker run --rm localhost:5000/hpc:hdf5 lstopo --version
lstopo 2.13.0
laptop:hpc-env $
```

Base image layers + one layer per package



- **link + run** dependencies only, in topological order
- Curiosity: layers are **independent** (unlike with Dockerfile)
- The config's `Env` comes from Spack's own environment modifications (`PATH` , …
- The full spec JSON goes in the image config, so `spack install` can use it

Layer reuse in the OCI registry

```
laptop:hpc-env $ # now the whole environment: hdf5 and its dependencies are already in the registry
laptop:hpc-env $ spack -e . buildcache push --without-build-dependencies --base-image ubuntu:26.04 --tag hpc-e
nv hpc-registry
==> Selected 32 specs to push to oci+http://localhost:5000/hpc
==> Checking for existing specs in the buildcache
==> 20 specs need to be pushed to localhost:5000/hpc
==> [ 1/20] Pushed pigz@2.8/afklka7: sha256:26a394fbc13c… (0.02s, 4.70 MB/s)
==> [ 2/20] Pushed libmd@1.1.0/yvvn7mu: sha256:5ee4c4f60d6a… (0.03s, 5.09 MB/s)
==> [ 3/20] Pushed tar@1.35/f5xe4px: sha256:71bbe27c867b… (0.02s, 58.93 MB/s)
==> [ 4/20] Pushed libbsd@0.12.2/7o3mv6c: sha256:c0d304da1601… (0.03s, 16.72 MB/s)
==> [ 5/20] Pushed zstd@1.5.7/63aruxk: sha256:ef23eb089d94… (0.02s, 112.02 MB/s)
==> [ 6/20] Pushed util-linux-uuid@2.41/clufswx: sha256:d732373f043f… (0.02s, 171.49 MB/s)
==> [ 7/20] Pushed sqlite@3.53.1/unjd5da: sha256:eb4e2ef470fc… (0.03s, 326.27 MB/s)
==> [ 8/20] Pushed readline@8.3/k25xiih: sha256:2de1a6e49ae8… (0.03s, 58.19 MB/s)
==> [ 9/20] Pushed openssl@3.6.1/33yozp5: sha256:e375bc5a0f8a… (0.04s, 206.04 MB/s)
: 31 lines omitted :
==> [19/20] Tagged py-mpi4py@4.1.1/tjrgxz6 as localhost:5000/hpc:py-mpi4py-4.1.1-tjrgxz6….spack
==> [20/20] Tagged py-h5py@3.16.0/qaefg7q as localhost:5000/hpc:py-h5py-3.16.0-qaefg7q….spack
==> The following 12 specs were skipped as they already exist in the buildcache:
    xz@5.8.3/hhjyyqy, libiconv@1.18/vbwvgwx, ncurses@6.6/ekvivpv, libxml2@2.15.3/ujlg2ua, ..., hdf5@1.14.6/jce
yzq7
    Use --force to overwrite them.
==> Tagged localhost:5000/hpc:hpc-env
laptop:hpc-env $ docker pull -q localhost:5000/hpc:hpc-env
localhost:5000/hpc:hpc-env
laptop:hpc-env $ docker run --rm localhost:5000/hpc:hpc-env python3 -c 'import h5py; print(h5py.version.info)'
Summary of the h5py configuration
---------------------------------

h5py    3.16.0
HDF5    1.14.6
Python  3.14.5 (main, Oct 14 2026, 14:31:30) [GCC 15.2.0]
sys.platform    linux
sys.maxsize     9223372036854775807
numpy   2.4.6
cython (built with) 3.2.4
numpy (built against) 2.4.6
HDF5 (built against) 1.14.6

laptop:hpc-env $ docker run --rm localhost:5000/hpc:hpc-env mpiexec -n 4 python3 -c 'from mpi4py import MPI; p
rint("rank", MPI.COMM_WORLD.rank)'
rank 1
rank 3
rank 0
rank 2
laptop:hpc-env $
```

Tags, layers and environment

```
laptop $ # the registry now has one tag per package, plus our two image tags
laptop $ skopeo list-tags --tls-verify=false docker://localhost:5000/hpc
{
    "Repository": "localhost:5000/hpc",
    "Tags": [
        "bzip2-1.0.8-cbtgjrh….spack",
        "expat-2.8.1-d462mhw….spack",
        "gcc-runtime-15.2.0-xm76mt3….spack",
        "gdbm-1.26-cq4dkuc….spack",
        "gettext-1.0-sle3ix4….spack",
        "hdf5",
        "hdf5-1.14.6-jceyzq7….spack",
        "hpc-env",
        "hwloc-2.13.0-ujctyey….spack",
        "libbsd-0.12.2-7o3mv6c….spack",
: 14 lines omitted :
        "py-numpy-2.4.6-evb4x7y….spack",
        "python-3.14.5-derywaq….spack",
        "python-venv-1.0-j76kyd4….spack",
        "readline-8.3-k25xiih….spack",
        "sqlite-3.53.1-unjd5da….spack",
        "tar-1.35-f5xe4px….spack",
        "util-linux-uuid-2.41-clufswx….spack",
        "xz-5.8.3-hhjyyqy….spack",
        "zlib-ng-2.3.3-g72d7i3….spack",
        "zstd-1.5.7-63aruxk….spack"
    ]
}
laptop $ # the hpc-env image: base image layers + one layer per package
laptop $ skopeo inspect --tls-verify=false docker://localhost:5000/hpc:hpc-env | jq '.Layers | length'
34
laptop $ # its config sets the environment, pointing into each package prefix (first entry shown)
laptop $ skopeo inspect --tls-verify=false --config docker://localhost:5000/hpc:hpc-env | jq -r '.config.Env[]
' | cut -d: -f1
PATH=/home/bot/lrz-demo/store/…/hdf5-1.14.6-jceyzq7/bin
ACLOCAL_PATH=/home/bot/lrz-demo/store/…/pkgconf-2.5.1-yvl6jpi/share/aclocal
CMAKE_PREFIX_PATH=/home/bot/lrz-demo/store/…/py-h5py-3.16.0-qaefg7q
LD_LIBRARY_PATH=/home/bot/lrz-demo/store/…/libfabric-2.5.1-itb4a2s/lib64
MANPATH=/home/bot/lrz-demo/store/…/pkgconf-2.5.1-yvl6jpi/share/man
MPICC=/home/bot/lrz-demo/store/…/mpich-5.0.1-xkilhym/bin/mpicc
MPICXX=/home/bot/lrz-demo/store/…/mpich-5.0.1-xkilhym/bin/mpicxx
MPIF77=/home/bot/lrz-demo/store/…/mpich-5.0.1-xkilhym/bin/mpif77
MPIF90=/home/bot/lrz-demo/store/…/mpich-5.0.1-xkilhym/bin/mpif90
PKG_CONFIG_PATH=/home/bot/lrz-demo/store/…/hdf5-1.14.6-jceyzq7/lib/pkgconfig
PYTHONPATH=/home/bot/lrz-demo/store/…/py-numpy-2.4.6-evb4x7y/lib/python3.14/site-packages
laptop $
```

What a runtime does with the image, by hand

```
laptop:rt $ # 1. download the image as plain files
laptop:rt $ skopeo copy -q --src-tls-verify=false docker://localhost:5000/hpc:hpc-env oci:img
laptop:rt $ cd img/blobs/sha256 && manifest=$(jq -r '.manifests[0].digest[7:]' ../../index.json)
laptop:rt $ config=$(jq -r '.config.digest[7:]' $manifest); jq '.layers | length' $manifest
34
laptop:rt $ # 2. untar all layers in order into one directory
laptop:rt $ mkdir ~/lrz-demo/work/rt/rootfs && for l in $(jq -r '.layers[].digest[7:]' $manifest); do tar -xzf
 $l -C ~/lrz-demo/work/rt/rootfs; done
laptop:rt $ cd ~/lrz-demo/work/rt && ls rootfs
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
laptop:rt $ ls rootfs/home/bot/lrz-demo/store/linux-x86_64_v3 | head -4
bzip2-1.0.8-cbtgjrhpwivtaewryhbkj6zxuukhoekx
expat-2.8.1-d462mhwofp3qarbz5celmkqikzukdqnk
gcc-runtime-15.2.0-xm76mt35elmqwrjdlibzhngqkiqnyq4p
gdbm-1.26-cq4dkucz57w6rd7gnkhgbzfqz4snldpy
laptop:rt $ jq -r '.config.Env[]' img/blobs/sha256/$config > image.env
laptop:rt $ # 3. run it unprivileged with the image's environment
laptop:rt $ bwrap --bind rootfs / --ro-bind /sys /sys --proc /proc --dev /dev --tmpfs /tmp env -i $(cat image.
env) python3 -c 'import h5py; print(h5py.version.info)'
Summary of the h5py configuration
---------------------------------

h5py    3.16.0
HDF5    1.14.6
Python  3.14.5 (main, Oct 14 2026, 14:31:30) [GCC 15.2.0]
sys.platform    linux
sys.maxsize     9223372036854775807
numpy   2.4.6
cython (built with) 3.2.4
numpy (built against) 2.4.6
HDF5 (built against) 1.14.6

laptop:rt $ bwrap --bind rootfs / --ro-bind /sys /sys --proc /proc --dev /dev --tmpfs /tmp env -i $(cat image.
env) mpiexec -n 4 python3 -c 'from mpi4py import MPI; print("rank", MPI.COMM_WORLD.rank)'
rank 1
rank 2
rank 3
rank 0
laptop:rt $
```

ON HPC SYSTEMS

OCI container images are compatible with other tools

RUNTIME	HOW
Apptainer	`apptainer build env.sif docker://registry/img:tag`
Enroot	`enroot import docker://registry#img:tag`
Charliecloud	`ch-image pull registry/img:tag`
Kubernetes, JupyterHub	containerd: as is

SAME REGISTRY, OTHER PURPOSE · RECORDED

The image is also a build cache

```
machine-c $ # another machine, no Docker needed: the registry is just a build cache
machine-c $ spack env create -d my-env ~/lrz-demo/envs/hpc/spack.lock
==> Created independent environment in: /home/bot/lrz-demo/machines/c/my-env
==> Activate with: spack env activate my-env
==> Updating view at /home/bot/lrz-demo/machines/c/my-env/.spack-env/view
machine-c $ spack -e my-env mirror add --unsigned hpc-registry oci+http://localhost:5000/hpc
machine-c $ spack -e my-env config add config:install_tree:root:~/lrz-demo/c
machine-c $ spack -e my-env install
[e] yc4n2pp glibc@2.43 /usr (0s)
[+] xm76mt3 gcc-runtime@15.2.0 /home/bot/lrz-demo/c/…/gcc-runtime-15.2.0-xm76mt3 (1s)
[+] yvvn7mu libmd@1.1.0 /home/bot/lrz-demo/c/…/libmd-1.1.0-yvvn7mu (0s)
[+] cbtgjrh bzip2@1.0.8 /home/bot/lrz-demo/c/…/bzip2-1.0.8-cbtgjrh (0s)
[+] aiceosz libffi@3.5.2 /home/bot/lrz-demo/c/…/libffi-3.5.2-aiceosz (0s)
[+] itb4a2s libfabric@2.5.1 /home/bot/lrz-demo/c/…/libfabric-2.5.1-itb4a2s (0s)
[+] clufswx util-linux-uuid@2.41 /home/bot/lrz-demo/c/…/util-linux-uuid-2.41-clufswx (0s)
[+] qzkk5ym libpciaccess@0.17 /home/bot/lrz-demo/c/…/libpciaccess-0.17-qzkk5ym (0s)
[+] g72d7i3 zlib-ng@2.3.3 /home/bot/lrz-demo/c/…/zlib-ng-2.3.3-g72d7i3 (0s)
[+] 63aruxk zstd@1.5.7 /home/bot/lrz-demo/c/…/zstd-1.5.7-63aruxk (0s)
[+] vbwvgwx libiconv@1.18 /home/bot/lrz-demo/c/…/libiconv-1.18-vbwvgwx (0s)
[+] yvl6jpi pkgconf@2.5.1 /home/bot/lrz-demo/c/…/pkgconf-2.5.1-yvl6jpi (0s)
[+] hhjyyqy xz@5.8.3 /home/bot/lrz-demo/c/…/xz-5.8.3-hhjyyqy (0s)
[+] 7o3mv6c libbsd@0.12.2 /home/bot/lrz-demo/c/…/libbsd-0.12.2-7o3mv6c (0s)
[+] afklka7 pigz@2.8 /home/bot/lrz-demo/c/…/pigz-2.8-afklka7 (0s)
[+] 33yozp5 openssl@3.6.1 /home/bot/lrz-demo/c/…/openssl-3.6.1-33yozp5 (1s)
[+] ujlg2ua libxml2@2.15.3 /home/bot/lrz-demo/c/…/libxml2-2.15.3-ujlg2ua (1s)
[+] f5xe4px tar@1.35 /home/bot/lrz-demo/c/…/tar-1.35-f5xe4px (1s)
[+] d462mhw expat@2.8.1 /home/bot/lrz-demo/c/…/expat-2.8.1-d462mhw (1s)
[+] 7kdghmr openblas@0.3.33 /home/bot/lrz-demo/c/…/openblas-0.3.33-7kdghmr (1s)
[+] ekvivpv ncurses@6.6 /home/bot/lrz-demo/c/…/ncurses-6.6-ekvivpv (1s)
[+] k25xiih readline@8.3 /home/bot/lrz-demo/c/…/readline-8.3-k25xiih (0s)
[+] ujctyey hwloc@2.13.0 /home/bot/lrz-demo/c/…/hwloc-2.13.0-ujctyey (0s)
[+] cq4dkuc gdbm@1.26 /home/bot/lrz-demo/c/…/gdbm-1.26-cq4dkuc (1s)
[+] unjd5da sqlite@3.53.1 /home/bot/lrz-demo/c/…/sqlite-3.53.1-unjd5da (1s)
[+] sle3ix4 gettext@1.0 /home/bot/lrz-demo/c/…/gettext-1.0-sle3ix4 (1s)
[+] xkilhym mpich@5.0.1 /home/bot/lrz-demo/c/…/mpich-5.0.1-xkilhym (2s)
[+] jceyzq7 hdf5@1.14.6 /home/bot/lrz-demo/c/…/hdf5-1.14.6-jceyzq7 (0s)
[+] derywaq python@3.14.5 /home/bot/lrz-demo/c/…/python-3.14.5-derywaq (2s)
[+] j76kyd4 python-venv@1.0 /home/bot/lrz-demo/c/…/python-venv-1.0-j76kyd4 (0s)
[+] tjrgxz6 py-mpi4py@4.1.1 /home/bot/lrz-demo/c/…/py-mpi4py-4.1.1-tjrgxz6 (0s)
[+] evb4x7y py-numpy@2.4.6 /home/bot/lrz-demo/c/…/py-numpy-2.4.6-evb4x7y (1s)
[+] qaefg7q py-h5py@3.16.0 /home/bot/lrz-demo/c/…/py-h5py-3.16.0-qaefg7q (0s)
==> Updating view at /home/bot/lrz-demo/machines/c/my-env/.spack-env/view
machine-c $
```

CONSEQUENCES

Archiving instead of building

You get

- **Deduplication**: images that share dependencies share layers, in the registry and in the runtime's local store
- **Incremental** pushes: only new builds are uploaded
- Building on the host is easier to troubleshoot
- No root needed
- The container registry serves both `spack install` and `docker run`

Things to be aware of

- You need to pick a base image with compatible glibc
- Use generic targets like `target=x86_64_v2` for CPU compat
- The **install paths**: `/home/me/spack/opt/…` ends up in the image. Use a neutral `install_tree: root: /opt/spack`.
- One architecture per tag (no multi-arch index)
- Some limitations to max number of layers

LIMITATION

How many layers can an image have?

Overlay-based runtimes mount every layer as an overlayfs `lowerdir`. Spack makes **one layer per package**, and large environments easily reach 150+ packages.

RUNTIME / STORAGE	MAX LAYERS
Docker, classic `overlay2` driver	**125**
Docker ≥ 29 (fresh install): containerd image store	499
Podman (containers/storage)	≈ 500
Apptainer, Enroot, Charliecloud	no limit: **flatten**

```
// moby: daemon/internal/layer/layer_store.go
// maxLayerDepth represents the maximum number of
// layers which can be chained together. 125 was
// chosen to account for the 127 max in some
// graphdrivers plus the 2 additional layers
// used to create a rwlayer.
const maxLayerDepth = 125
```

The kernel's own overlayfs limit is about 500 lower layers, which is what containerd and Podman run into.

Measured with rootless Docker 29.8 on Linux 7.0, see the next slide.

LAYER LIMIT · RECORDED

Classic `overlay2`: 125 layers

```
laptop $ # synthetic images: ubuntu:26.04 (2 layers) + N tiny layers
laptop $ docker info --format '{{.Driver}}'
overlay2
laptop $ skopeo inspect --tls-verify=false docker://localhost:5000/layers:126 | jq '.Layers|length'
126
laptop $ docker pull -q localhost:5000/layers:125 && docker run --rm localhost:5000/layers:125 echo ok
localhost:5000/layers:125
ok
laptop $ docker pull -q localhost:5000/layers:126
failed to register layer: max depth exceeded
laptop $
```

containerd image store: 499 layers

```
laptop $ # same synthetic images, Docker 29 default: containerd image store
laptop $ docker info --format '{{.Driver}} {{.DriverStatus}}'
overlayfs [[driver-type io.containerd.snapshotter.v1]]
laptop $ docker pull -q localhost:5000/layers:126 && docker run --rm localhost:5000/layers:126 echo ok
localhost:5000/layers:126
ok
laptop $ docker pull -q localhost:5000/layers:499 && docker run --rm localhost:5000/layers:499 echo ok
localhost:5000/layers:499
ok
laptop $ docker pull -q localhost:5000/layers:500 && docker run --rm localhost:5000/layers:500 echo ok 2>&1 | gr
ep -o 'err: .*'
localhost:5000/layers:500
err: invalid argument
laptop $
```

LAYER LIMIT

Working around layer limits

- **Smaller images**: push subset of environment
  `spack buildcache push --tag x mirror hdf5 python`
- **Flattening runtimes** (Apptainer, Enroot, Charliecloud) don't have the limit at all
- **Request** me to implement a form of layer squashing
- Use some externals from base image; that's again the externals ↔ base image trade-off.

ASIDE

Containers don't solve all the runtime problems

The container's MPI must talk to the **host's** launcher (Slurm/PMIx) and fabric (libfabric/UCX providers, drivers).

- **Hybrid**: an MPI inside the image that is compatible with the host's process manager and fabric, launched with `srun` / `mpiexec` from outside
- **Bind**: mount the host's MPI over the container's. Requires ABI compatibility: the MPICH family (MPICH, Intel MPI, Cray MPICH) shares an ABI; MPI 5.0 standardizes one.
- GPU drivers / `libcuda.so` are outside of Spack's scope, they are from the system.

CHOOSING

containerize vs. buildcache push

	SPACK CONTAINERIZE	SPACK BUILDCACHE PUSH --BASE-IMAGE
Builds happen	inside a container build	on the host, in CI, in a constainer, ...
Needs	Docker / Podman / Apptainer, often root	a registry; no privileges
glibc / externals	**controlled** by the builder image	**inherited** from the host; pick a compatible base
Layers	one "fat" layer	one per Spack package
Change one package	rebuild (or re-fetch) everything	push one layer
Customization	anything a Dockerfile can do	no, would require `FROM spack-image` Dockerfile
Doubles as build cache	no	**yes**

4

Hands-on

Spack tutorial: **Binary Caches**. Install from a build cache, run a local registry, push, and turn an environment into a runnable image.

HANDS-ON · ON YOUR OWN MACHINE

Two shells

Shell 1: a local registry

```
$ docker run --rm -p 5000:5000 registry
```

Leave it running. Any OCI registry works the same way; a local one needs no credentials (`oci+http://localhost:5000/…`).

Shell 2: Spack

```
$ git clone --depth=2 --branch=releases/v1.2 \
    https://github.com/spack/spack
$ . spack/share/spack/setup-env.sh
```

Then follow **spack-tutorial.readthedocs.io → Binary Caches Tutorial**:

- install from a build cache (relocation)
- push to `localhost:5000`
- `--base-image`: why glibc matters
- `--tag`: environments as images

Thank you!

Questions?

Harmen Stoppels · github.com/haampie



What's new in Spack

v1.2 came out on 21 June 2026

v1.3 is planned for November 2026.

SPACK V1.2 · JUNE 2026

Highlights of v1.2

New installer (default)

Runs independent builds in parallel under a jobserver for dynamic parallelism. New TUI: `v` follows a build's log, `n` / `p` switches to the next/previous one.

Concretization groups

Groups of specs in an environment. Each group can specify local config and can `needs:` other groups.

Concretization cache

On by default: after `spack spec hdf5`, a `spack install hdf5` doesn't solve again.

SBOMs

Every install gets an SPDX 2.3 SBOM in `$prefix/.spack/sbom`.

`spack isolate`

Avoid writing to `~/.spack` (will be improved further in v1.3).

Sandboxing (experimental)

Linux Landlock, no root or namespaces needed. A build can write only to its stage and prefix.

Concretization groups

Bootstrap a compiler: `needs:`

```
spack:
  specs:
  - group: compiler
    specs: [gcc@15.2]

  - group: apps
    needs: [compiler]
    specs:
    - hdf5 %gcc@15.2
    - libtree %gcc@15.2
```

The groups listed in `needs:` are concretized first, and their specs can be reused.

CPU and CUDA builds: `override:`

```
spack:
  specs:
  - group: cpu
    specs: [gromacs, lammps]

  - group: cuda
    specs: [gromacs, lammps]
    override:
      packages:
        all:
          prefer: ["+cuda cuda_arch=90"]
  view: false
```

The override is a config scope for that group only.

v1.2 improvements for build caches

Choose what a mirror is used for

```
mirrors:
  site:
    url: oci://…
    include_binary: ["%gcc"]
    exclude_binary: ["^mpich"]
```

This applies both when installing from the mirror and when pushing to it.

- `spack concretize` marks specs that are not installed yet but are in a build cache with `[b]`
- The new installer writes the database faster, which you notice when installing many packages from a build cache

Isolation of Spack itself

Spack is currently "installed" with git clone, and modifies files inside its dir, like `$spack/opt/spack`, `$spack/var/spack/environments`, GPG keys, licenses…

Shared Spack: user data goes into XDG locations:

```
~/.config/spack       # config (was ~/.spack)
~/.local/share/spack  # installs, envs, keys
~/.local/state/spack  # misc cache, repos
~/.cache/spack        # caches
```

What this enables

- `pip install spack`, distro packages
- Single Spack instance can be installed globally and used by multiple users

Existing clones

After a `git pull`, Spack asks before it copies environments, keys and licenses.

Usages: variants on dependency edges

Some options change **how a dependency is used**, but not the dependency itself:

- when *using* gcc, enable ASan
- when *using* gcc, write SARIF diagnostics

As a variant on `gcc`, this changes gcc's hash: gcc gets rebuilt, and every package in the DAG has to agree on the value.

Spec syntax (subject to change)

```
zlib-ng %[usages=+sarif] gcc
foo %[usages=+sarif sanitizers=asan] gcc
foo ^[deptypes=link usages=+sarif] mpich
```

Declared by the dependency

```
class Gcc(Package):
    usage("sarif", default=False,
          description="emit SARIF diagnostics")
```

Similar to `variant()`

SPACK V1.3 · PLANNED FOR NOVEMBER 2026

Deprecations with a reason and a severity

Package authors

```
class Openssl(Package):
    version("3.0.7", sha256="…")
    version("1.1.1t", sha256="…")

    deprecated("@1.1.1t", reason="vuln",
               severity="high",
               labels=["CVE-2023-0286"])
```

- `reason`: `vuln`, `rename`, `retired` or `unspecified`
- `severity`: `low` … `critical`, following CVSS ratings
- Replaces boolean `version(…, deprecated=True)` which was per version.

Sites and users

```
packages:
  all:
    deprecation:
      allow:
      - reason: [rename, retired, unspecified]
        severity: low
      - reason: vuln
        severity: none
```

- Not every CVE is as critical; it depends on the context.